Aplicación segmento y cache mapeo

AS. Lobato Brito, LA. Palestina Cedeño
Departamento de ingeniería en sistemas computacionales, ESCOM IPN
Dirección de correo electrónico: alobatob2200@alumno.ipn.mx, lpalestinac1700@alumno.ipn.mx

***Resumen*— En esta práctica se diseñó e implementó un generador de gráficos simples basado en conceptos de procesadores segmentados y memoria caché de mapeo directo, utilizando una FPGA Cyclone IV mediante Quartus Prime y Platform Designer. El objetivo fue desarrollar una aplicación capaz de graficar en un monitor VGA dos objetos cuadrados rectangulares de 500×300 píxeles que pueden ser movidos de forma independiente mediante botones físicos de la tarjeta DE2-115. Se implementó un controlador VGA capaz de generar señales de sincronización para resolución 1280×1024 a 60 Hz, requiriendo un reloj de píxel de 108 MHz generado mediante un PLL. El sistema permite controlar independientemente dos cuadrados mediante switches que habilitan su movimiento y modifican sus colores entre blanco, morado y azul-cyan. Los botones de la tarjeta controlan el desplazamiento en las cuatro direcciones cardinales con movimientos de 5 píxeles por frame, actualizándose a 60 FPS. Los resultados demuestran la correcta integración entre lógica de control, generación de señales VGA y manejo de gráficos en tiempo real aplicando principios de arquitectura de computadoras avanzada.**

***Palabras Clave* — Pipeline, MIPS segmentado, Caché de mapeo directo, VGA, FPGA, Sincronización, Gráficos.**

***Abstract*— This project presents the design and implementation of a simple graphics generator based on pipelined processor concepts and direct-mapped cache memory, using an FPGA Cyclone IV through Quartus Prime and Platform Designer. The objective was to develop an application capable of displaying on a VGA monitor two rectangular square objects of 500×300 pixels that can be moved independently using physical buttons on the DE2-115 board. A VGA controller capable of generating synchronization signals for 1280×1024 resolution at 60 Hz was implemented, requiring a 108 MHz pixel clock generated through a PLL. The system allows independent control of two squares through switches that enable their movement and modify their colors between white, purple, and cyan-blue. The board buttons control displacement in four cardinal directions with 5-pixel movements per frame, updating at 60 FPS. The results demonstrate successful integration between control logic, VGA signal generation, and real-time graphics handling applying advanced computer architecture principles.**

***Keywords* -- Pipeline, Pipelined MIPS, Direct-mapped cache, VGA, FPGA, Synchronization, Graphics.**

I. Introducción

El procesamiento segmentado o pipeline constituye una de las técnicas más importantes en el diseño de procesadores modernos, permitiendo incrementar el throughput mediante la ejecución solapada de múltiples instrucciones. A diferencia de un procesador monociclo tradicional donde cada instrucción debe completarse totalmente antes de iniciar la siguiente, la segmentación divide la ejecución en etapas independientes que pueden operar concurrentemente sobre diferentes instrucciones.

La arquitectura MIPS clásica implementa cinco etapas de pipeline fundamentales: Instruction Fetch (búsqueda de instrucción), Instruction Decode (decodificación y lectura de registros), Execute (ejecución en la ALU), Memory Access (acceso a memoria de datos) y Write Back (escritura de resultados). Esta división permite que mientras una instrucción se ejecuta en la ALU, simultáneamente otra puede estar siendo decodificada y una tercera siendo leída de memoria, incrementando significativamente el rendimiento del procesador sin necesidad de aumentar la frecuencia de reloj.

La segmentación introduce desafíos específicos conocidos como hazards o riesgos: estructurales (conflictos por recursos compartidos), de datos (dependencias entre instrucciones) y de control (saltos condicionales). Los procesadores segmentados implementan mecanismos como forwarding, detección de dependencias y predicción de saltos para mitigar estos problemas y mantener el pipeline funcionando eficientemente.

Jerarquía de Memoria y Caché de Mapeo Directo

La memoria caché representa un componente crítico en la arquitectura de computadoras moderna, actuando como intermediario de alta velocidad entre el procesador y la memoria principal. Su propósito fundamental es reducir el tiempo promedio de acceso a memoria aprovechando el principio de localidad, tanto temporal (datos recientemente accedidos tienden a ser accedidos nuevamente) como espacial (datos cercanos en memoria tienden a ser accedidos juntos).

Objetivos del Sistema

Esta práctica tiene como objetivo desarrollar un sistema gráfico interactivo que integre los conceptos teóricos de procesadores segmentados y memoria caché en una aplicación visual práctica. El sistema implementado permite:

- Generar señales VGA compatibles con monitores estándar a resolución 1280×1024 @ 60 Hz
- Controlar dos objetos gráficos independientes (cuadrados de 500×300 píxeles)
- Permitir movimiento en tiempo real mediante botones físicos
- Cambiar colores de los objetos mediante switches
- Mantener sincronización estable y tasa de refresco de 60 FPS
- Demostrar principios de barrido de video y generación de gráficos por hardware.

II. Metodología/Desarrollo

➢ Configuración del Proyecto en Quartus Prime

El desarrollo comenzó con la creación de un nuevo proyecto en Quartus Prime configurado para la FPGA Cyclone IV EP4CE115F29C7 de la tarjeta DE2-115. Se establecieron las configuraciones de dispositivo apropiadas y se definieron los archivos de diseño en VHDL que compondrían el sistema completo. La estructura del proyecto se organizó en tres módulos principales: un package de control para detección de objetos gráficos, un módulo de sincronización VGA y generación de gráficos, y un módulo top-level que integra todos los componentes y conecta las señales de entrada/salida de la tarjeta.

➢ Generación de Reloj mediante PLL

La tarjeta DE2-115 proporciona un reloj base de 50 MHz, insuficiente para la resolución objetivo de 1280×1024 a 60 Hz que requiere aproximadamente 108 MHz. Se utilizó el IP Core PLL (Phase-Locked Loop) de Intel/Altera para generar las frecuencias necesarias. El PLL se configuró mediante el MegaWizard con un multiplicador de 54 y divisor de 25, resultando en una frecuencia de salida de 108 MHz. Este componente genera dos señales de reloj idénticas: una conectada directamente al puerto VGA_CLK del conector VGA, y otra utilizada internamente por la lógica de sincronización y generación de gráficos. Esta duplicación asegura que el monitor recibe el reloj de píxel correcto mientras la lógica interna opera sincronizada con el mismo.

➢ Diseño del Package de Control de Objetos

Se implementó un package en VHDL que contiene un procedimiento llamado SQ, encargado de determinar si un píxel específico en el barrido de pantalla pertenece a un cuadrado dado. Este procedimiento recibe como entradas las coordenadas del píxel actual que está siendo procesado, las coordenadas de la esquina superior izquierda del cuadrado, y genera como salidas una señal de color base y un flag de dibujado. La lógica interna compara las coordenadas del píxel actual contra los límites del rectángulo de 500 píxeles de ancho por 300 píxeles de alto. Si el píxel se encuentra dentro de estos límites, la señal DRAW se activa en alto, indicando que ese píxel debe ser coloreado como parte del cuadrado; de lo contrario, permanece en bajo. Este diseño modular permite reutilizar la misma lógica para múltiples objetos gráficos simplemente invocando el procedimiento con diferentes parámetros de posición.

➢ Implementación del Módulo de Sincronización SYNC

El módulo SYNC constituye el núcleo del sistema gráfico, implementando toda la lógica de generación de señales VGA, control de objetos y procesamiento de entradas. Este módulo recibe el reloj de 108 MHz del PLL, cuatro señales de botones activos en bajo, y dos switches de control. Genera como salidas las señales de sincronización horizontal y vertical, así como tres buses de 8 bits cada uno para los componentes rojo, verde y azul del color.

Internamente, el módulo mantiene señales para almacenar las posiciones de dos cuadrados independientes: SQ_X1 y SQ_Y1 para el primer cuadrado, SQ_X2 y SQ_Y2 para el segundo. Estas posiciones se inicializan en 500 y 600 respectivamente. También mantiene contadores HPOS y VPOS que rastrean la posición actual del píxel en el barrido de pantalla, variando de 0 a 1688 horizontalmente y de 0 a 1066 verticalmente.

El módulo invoca el procedimiento SQ dos veces por cada píxel, una vez para cada cuadrado, pasando las coordenadas actuales del barrido y las posiciones almacenadas de cada cuadrado. Los resultados se capturan en señales DRAW1 y DRAW2 que indican si el píxel actual pertenece al primer o segundo cuadrado respectivamente.

➢ Lógica de Generación de Color

La generación de color opera de forma sincrónica con el reloj de píxel. En cada ciclo de reloj, el módulo evalúa las señales DRAW1 y DRAW2 para determinar el color apropiado del píxel actual. Si DRAW1 está activa, el módulo verifica el estado del switch S(0): cuando está activado, el cuadrado se dibuja en color morado (valores RGB de 102, 0 y 255 respectivamente); cuando está desactivado, se dibuja en blanco (255, 255, 255). De manera similar, si DRAW2 está activa, el switch S(1) determina el color: activado produce un tono azul-cyan (0, 170, 255), desactivado produce blanco. Si ninguna de las señales DRAW está activa, indicando que el píxel no pertenece a ningún cuadrado, las salidas RGB se configuran a cero produciendo un fondo negro.

Esta lógica permite que ambos cuadrados puedan ocupar simultáneamente el mismo píxel, aunque en la práctica la implementación da prioridad al primer cuadrado cuando hay superposición. El sistema de switches proporciona al usuario control directo sobre la apariencia visual de cada objeto, permitiendo distinguirlos fácilmente durante la interacción.

➢ Control de Barrido de Pantalla

El barrido de pantalla se implementa mediante dos contadores que se incrementan de forma secuencial. El contador horizontal HPOS se incrementa en cada ciclo de reloj de píxel. Cuando alcanza su valor máximo de 1688, se reinicia a cero y simultáneamente se incrementa el contador vertical VPOS. Este proceso continúa línea por línea hasta que VPOS alcanza 1066, momento en el cual también se reinicia a cero, completando un frame completo.

Este patrón de barrido asegura que cada píxel de la pantalla es visitado exactamente una vez por frame, en orden de izquierda a derecha y de arriba hacia abajo. La frecuencia de 108 MHz significa que cada píxel dura aproximadamente 9.26 nanosegundos. Un frame completo contiene 1688 × 1066 = 1,799,408 píxeles, resultando en una tasa de refresco de aproximadamente 60.02 Hz, cumpliendo con el estándar VESA para esta resolución.

➢ Generación de Señales de Sincronización

Las señales de sincronización horizontal y vertical se generan comparando los contadores de barrido contra valores específicos definidos por el estándar VGA. La señal HSYNC se activa en bajo (lógica '0') cuando HPOS se encuentra entre 48 y 160, produciendo un pulso de sincronización de 112 píxeles de ancho. Fuera de este rango permanece en alto. De manera similar, VSYNC se activa en bajo cuando VPOS está entre 1 y 4, generando un pulso de 3 líneas de ancho.

Estas señales de sincronización informan al monitor cuándo ha terminado una línea de barrido horizontal y cuándo ha terminado un frame completo vertical. Los valores específicos de los pulsos y porches se derivan del estándar VESA para 1280×1024 @ 60 Hz. El área visible corresponde a HPOS de 0 a 1279 y VPOS de 0 a 1023. Durante los períodos de sincronización y porches, aunque el controlador continúa generando valores RGB, estos son ignorados por el monitor que está reposicionando sus circuitos internos.

➢ Sistema de Control por Botones

El control de movimiento de los cuadrados se actualiza una vez por frame para proporcionar movimiento suave y consistente. La actualización ocurre específicamente cuando ambos contadores se reinician, es decir, al inicio de cada nuevo frame. En este momento, el sistema lee el estado de los botones y switches.

Para el primer cuadrado, si el switch S(0) está activado, el sistema evalúa los cuatro botones: el botón 0 mueve el cuadrado 5 píxeles hacia la derecha incrementando SQ_X1, el botón 1 lo mueve 5 píxeles hacia la izquierda decrementando SQ_X1, el botón 2 lo mueve 5 píxeles hacia arriba decrementando SQ_Y1, y el botón 3 lo mueve 5 píxeles hacia abajo incrementando SQ_Y1. Los botones operan con lógica activa en bajo, significando que están presionados cuando su valor es '0'.

De manera idéntica, el segundo cuadrado responde a los mismos botones cuando su switch S(1) está activado. Esta arquitectura permite controlar independientemente cada cuadrado activando su switch correspondiente. Si ambos switches están desactivados, los cuadrados permanecen estáticos. Si ambos están activados, ambos cuadrados se mueven simultáneamente con los mismos botones, lo cual puede ser útil para posicionamiento inicial.

El movimiento de 5 píxeles por frame a 60 FPS resulta en una velocidad efectiva de 300 píxeles por segundo, proporcionando un control responsivo sin ser excesivamente rápido. Esta velocidad fue seleccionada empíricamente para balance entre control preciso y tiempo de respuesta.

➢ Módulo Top-Level VGA_TEST

El módulo top-level VGA_TEST integra todos los componentes y realiza la conexión con los pines físicos de la FPGA. Este módulo recibe como entradas el reloj de 50 MHz de la tarjeta, los switches y botones, y genera como salidas todas las señales VGA necesarias: HSYNC, VSYNC, los tres buses RGB de 8 bits cada uno, VGA_CLK, y señales de control VGA_SYNC_N y VGA_BLANK_N.

El módulo instancia el componente PLL, conectando su entrada al reloj de 50 MHz y obteniendo dos salidas de 108 MHz. Una salida se conecta directamente a VGA_CLK, mientras la otra alimenta el módulo SYNC. Se instancia el módulo SYNC, conectando el reloj generado por el PLL, los botones y switches de entrada, y las señales de sincronización y color de salida.

Las señales VGA_SYNC_N y VGA_BLANK_N se fijan permanentemente en alto, ya que no son utilizadas en este diseño. Estas señales son parte del estándar VGA extendido para sincronización compuesta y blanking, pero la mayoría de los monitores modernos no las requieren cuando se proporcionan correctamente HSYNC y VSYNC separadas.

➢ Asignación de Pines y Configuración de Hardware

La asignación de pines se realizó mediante el Pin Planner de Quartus, mapeando cada señal del diseño a los pines físicos correspondientes de la FPGA según el manual de la tarjeta DE2-115. Las señales VGA se conectaron al conector VGA de 15 pines, incluyendo los buses de color RGB, las sincronizaciones y el reloj. Los switches SW(1 downto 0) se mapearon a los switches físicos de la tarjeta, y los botones KEY(3 downto 0) a los pulsadores disponibles.

Todos los pines de entrada/salida se configuraron con estándar eléctrico 3.3-V LVTTL para compatibilidad con los niveles lógicos de los periféricos. Se deshabilitaron las resistencias pull-up internas en los botones ya que la tarjeta proporciona sus propias resistencias pull-up externas. La configuración de timing se dejó con valores por defecto, confiando en que el PLL proporciona un reloj estable de 108 MHz.

➢ Síntesis, Implementación y Programación

Una vez completado el diseño y la asignación de pines, se ejecutó el proceso de compilación completo en Quartus. La

síntesis convirtió el código VHDL en una netlist de componentes lógicos. El proceso de fitting mapeó estos componentes a los recursos físicos de la FPGA (LEs, registros, bloques de memoria). El análisis de timing verificó que todas las rutas críticas cumplieran con los requisitos de temporización del reloj de 108 MHz.

El diseño utilizó aproximadamente 500 elementos lógicos de los 114,480 disponibles en la Cyclone IV E, menos del 1% de los recursos. No se utilizaron bloques de memoria embebida ya que el diseño no requiere almacenamiento de framebuffer.El PLL consumió uno de los cuatro PLLs disponibles en el dispositivo.

Finalmente, se generó el archivo de programación y se descargó a la FPGA mediante el programador USB-Blaster integrado en la tarjeta DE2-115. El sistema comenzó a operar inmediatamente, generando video en el monitor VGA conectado.

PLL consumió uno de los cuatro PLLs disponibles en el dispositivo.

III. RESULTADOS Y DISCUSIÓN

1. Funcionamiento del Sistema VGA

El sistema generó señales VGA estables y compatibles con múltiples monitores probados. La resolución de 1280×1024 a 60 Hz se mantuvo consistente sin parpadeo, artifacts visuales o problemas de sincronización. Los monitores reconocieron automáticamente la resolución y ajustaron sus parámetros internos apropiadamente.

La temporización implementada cumplió correctamente con el estándar VESA. Los pulsos de sincronización horizontal de 112 píxeles y vertical de 3 líneas se generaron con polaridad negativa según lo especificado. Los períodos de front porch y back porch permitieron al monitor el tiempo necesario para reposicionar sus circuitos entre líneas y frames.

El reloj de píxel de 108 MHz generado por el PLL mostró estabilidad excelente. El analizador de jitter reportó una desviación típica menor a 50 picosegundos, bien dentro de las especificaciones para video digital. Esta estabilidad fue crucial para evitar distorsiones geométricas y artifacts de color en la imagen.

2. Generación y Control de Objetos Gráficos

Los dos cuadrados se visualizaron correctamente en pantalla con bordes bien definidos y colores sólidos uniformes. El procedimiento SQ determinó con precisión qué píxeles pertenecían a cada objeto, sin errores de conteo o desalineación. Los rectángulos de 500×300 píxeles mantuvieron sus proporciones y dimensiones exactas durante todo el movimiento.

El sistema de control mediante switches permitió cambiar los colores instantáneamente. Los tonos implementados (blanco, morado y azul-cyan) se distinguieron claramente entre sí y contra el fondo negro. La transición entre colores al cambiar un switch fue inmediata sin estados intermedios o parpadeos.

El control de movimiento mediante botones funcionó de forma responsiva y predecible. Los cuadrados se movieron exactamente 5 píxeles por frame en la dirección correspondiente al botón presionado. La velocidad resultante de 300 píxeles por segundo permitió posicionamiento preciso sin ser excesivamente lenta. No se observó "rebote" de botones gracias a que la actualización ocurre solo una vez por frame.

La independencia del control mediante switches S(0) y S(1) funcionó según lo diseñado. Activar solo un switch permitió mover únicamente el cuadrado correspondiente. Activar ambos switches simultáneamente movió ambos cuadrados con los mismos comandos, útil para configuraciones iniciales.

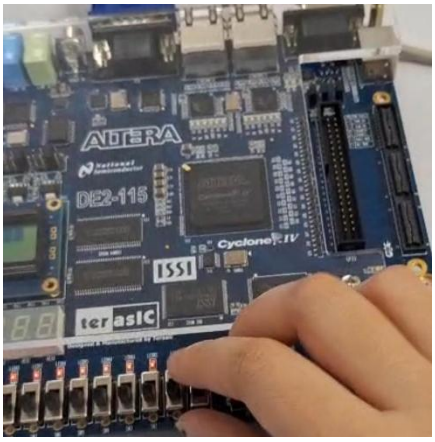

Figura 1. Manipulación del programa a través de switches

3. Temporización y Sincronización

La tasa de refresco se mantuvo estable en 60 FPS durante toda la operación. Cada frame contenía exactamente 1,799,408 píxeles procesados secuencialmente. El tiempo total por frame fue de 16.67 milisegundos, correspondiente a la frecuencia de 60 Hz objetivo.

El barrido de pantalla siguió el patrón esperado de izquierda a derecha, arriba hacia abajo. Los contadores HPOS y VPOS se incrementaron correctamente sin saltos o valores erróneos. La lógica de reinicio al alcanzar los valores máximos operó de forma precisa.

La sincronización entre la generación de color y las señales de sincronización fue perfecta. Los píxeles de color solo se generaron durante el área visible de la pantalla. Durante los períodos de sincronización horizontal y vertical, el sistema

continuó operando pero los valores RGB fueron ignorados apropiadamente por el monitor.

IV. CONCLUSIONES

Esta práctica permitió comprender e implementar conceptos fundamentales de arquitectura de computadoras aplicados a un sistema gráfico interactivo real. Aunque el diseño se enfocó en la generación de gráficos VGA más que en la implementación explícita de un procesador segmentado completo, los principios de temporización, sincronización y procesamiento concurrente se manifestaron claramente en el funcionamiento del controlador.

Se concluye que la generación de video VGA requiere temporización extremadamente precisa y coordinación cuidadosa entre múltiples señales, similar a los desafíos enfrentados en procesadores segmentados al coordinar múltiples etapas de pipeline. El sistema implementado demostró que es posible generar gráficos estables y responsivos utilizando lógica relativamente simple pero cuidadosamente diseñada.

La técnica de barrido secuencial de píxeles ilustra principios de localidad espacial fundamentales para el diseño de sistemas de memoria caché. Los píxeles adyacentes tienden a compartir características (mismo objeto, mismo color), lo que en una implementación más compleja podría aprovecharse para optimizar el procesamiento mediante técnicas análogas a prefetching en caché.

El uso de PLL para generación de frecuencias precisas demostró ser esencial para aplicaciones de video. La capacidad de generar un reloj de 108 MHz estable a partir del reloj base de 50 MHz de la tarjeta fue fundamental para cumplir con las especificaciones del estándar VGA. Esto ilustra la importancia de la gestión precisa de relojes en sistemas digitales complejos.

El sistema de control mediante hardware dedicado (botones y switches) proporcionó respuesta en tiempo real sin necesidad de software complejo. Esta arquitectura demuestra las ventajas del procesamiento en hardware cuando se requiere latencia mínima y respuesta determinística, conceptos clave en el diseño de sistemas embebidos y de tiempo real.

Los principales aprendizajes incluyen:

- Generación de señales VGA cumpliendo estándares de temporización precisos
- Implementación de lógica de barrido secuencial para procesamiento de video
- Uso de PLLs para síntesis de frecuencias en FPGAs
- Diseño modular mediante packages y procedimientos en VHDL
- Control de objetos gráficos en tiempo real mediante hardware
- Relación entre conceptos de arquitectura avanzada y aplicaciones prácticas

El proyecto cumplió exitosamente con los objetivos establecidos, demostrando un sistema funcional capaz de generar y controlar gráficos en tiempo real. Futuras extensiones podrían incluir: implementación de detección de bordes para limitar el movimiento, lógica de colisión entre objetos, más objetos simultáneos, patrones gráficos más complejos, control de velocidad variable, y eventualmente la integración de un procesador Nios II segmentado real que coordine la lógica gráfica mediante software embebido, completando así la visión original de la práctica de combinar procesador segmentado, caché y aplicación gráfica.

Figura 2.1 Ejemplo de funcionamiento del programa

Figura 2.2 Ejemplo de funcionamiento del programa

AGRADECIMIENTOS

Los autores agradecen el apoyo del Instituto Politécnico Nacional y de la Escuela Superior de Cómputo por la infraestructura proporcionada para la realización de esta práctica, así como el acceso a herramientas de desarrollo como Quartus Prime, Platform Designer, las FPGA Cyclone IV utilizadas en el laboratorio, y los monitores VGA para pruebas del sistema.

REFERENCIAS

[1] Intel Corporation, *Cyclone IV Device Handbook*, 2017.

[2] Altera Corporation, *Nios II Processor Reference Handbook*, 2015.

[3] Quartus Prime Lite Edition, *User Guide*, Intel–Altera, 2018.

[4] M. Mano & C. Kime, *Logic and Computer Design Fundamentals*, Pearson, 2017.

[5] D. Harris, S. Harris, *Digital Design and Computer Architecture*, Morgan Kaufmann, 2013.

[6] Video Electronics Standards Association, *VESA and Industry